

Kasutatud tehnoloogiate dokumentatsioon

1. Ülevaade

Projekti arendamisel kasutatakse mobiilirakenduste arendamiseks tehnoloogiaid Ionic 8.8, Capacitor 8.5 ja TypeScripti. Kasutajaliidese loomisel kasutatakse Ionicu Web Components'e ning projekt ei kasuta Reacti.

TypeScript on programmeerimiskeel, mis põhineb JavaScriptil ja lisab sellele staatilise tüübisüsteemi. TypeScripti kood teisendatakse JavaScriptiks, mistõttu saab seda kasutada veebitehnoloogiatel põhinevate rakenduste arendamisel. TypeScript aitab suuremates projektides vähendada tüüpvigu ning muudab koodi arusaadavamaks ja paremini hooldatavaks.

Ionic on kasutajaliidese raamistik, mille abil saab veebitehnoloogiaid kasutades luua mobiilirakendusi. Ionic pakub valmis kasutajaliidese komponente, näiteks nuppe, sisestusvälju, kaarte ja loendeid. Komponendid põhinevad Web Components'i tehnoloogial ning neid saab kasutada ka ilma Reactita.

Capacitor toimib sillana veebirakenduse ja mobiilseadme natiivsete funktsioonide vahel. Selle abil saab Ionicu rakenduse pakkida Androidi või iOS-i rakenduseks ning kasutada seadme võimalusi, millele tavalisel veebilehel otseselt ligi ei pääse.

2. TypeScripti võrdlus tuttavate programmeerimiskeeltega

TypeScript sarnaneb süntaksi poolest kõige rohkem JavaScriptiga, kuna TypeScript on JavaScripti laiendus. Samuti on TypeScriptis olemas mitmed tuttavad programmeerimise põhimõtted, näiteks muutujad, funktsioonid, tingimuslauseid, tsüklid, klassid ja objektid.

Võrreldes Pythoniga on oluline erinevus tüüpide käsitlemisel. Python on dünaamiliselt tüübitud keel, samas kui TypeScript võimaldab kasutada staatilist tüübisüsteemi. Näiteks saab TypeScriptis määrata, et muutuja peab sisaldama ainult teksti või ainult numbrit. See võimaldab avastada osa vigadest juba arendamise ajal, enne rakenduse käivitamist.

TypeScripti eelis Pythoniga võrreldes selle projekti puhul on eelkõige see, et TypeScript on loodud JavaScripti ökosüsteemi jaoks. Kuna Ionic põhineb veebitehnoloogiatel, sobib TypeScript Ionicu rakenduste arendamiseks hästi. Pythonit kasutatakse rohkem näiteks taustarakenduste, andmetöötamise ja skriptide kirjutamiseks.

3. Projekti seisukohalt olulised keerukamad omadused

3.1. Staatiline tüübisüsteem

Üks TypeScripti olulisemaid omadusi on staatiline tüübisüsteem. Muutujatele, funktsioonide parameetritele ja tagastusväärtustele saab määrata tüübid. Näiteks võib funktsioon eeldada, et talle antakse tekst või number.

Tüübid on mobiilirakenduse puhul kasulikud, sest rakenduses liigub erinevate komponentide vahel palju andmeid. Kui andmetüüp on vale, võib TypeScript anda sellest arendamise ajal märku. See aitab vähendada olukordi, kus rakendus töötab alles käivitamisel vigaselt.

Lisaks saab TypeScriptis kasutada `interface`-e, millega saab kirjeldada objektide struktuuri. See on kasulik näiteks kasutajaandmete või API-st saadavate andmete kirjeldamisel.

3.2. Asünkroonne programmeerimine

Teine projekti seisukohalt oluline omadus on asünkroonne programmeerimine. TypeScript toetab JavaScripti `Promise`-e ning `async` ja `await` märksõnu.

Mobiilirakenduses tuleb sageli oodata mõne tegevuse lõpetamist, näiteks andmete laadimist, API päringu vastust või seadme funktsiooniga suhtlemist. `async/await` võimaldab sellist koodi kirjutada lihtsamalt ja loetavamalt.

Asünkroonne programmeerimine on oluline ka Capacitori puhul, sest mitmed seadmfunktsioonid ja pluginad võivad töötada asünkroonselt. Rakendus saab samal ajal kasutajaliidest edasi kuvada ega pea kogu rakendust ühe tegevuse lõppemiseni peatama.

4. Seos mobiilirakenduste arendusega

Ionicu kasutamise peamine eesmärk selles projektis on võimaldada mobiilirakenduse loomist tuttavate veebitehnoloogiate abil. Kasutajaliidese jaoks kasutatakse HTML-i, CSS-i ja TypeScripti ning Ionic pakub nende kõrvale mobiilirakendustele sobivaid valmis komponente.

Ionicuga loodud rakendus töötab veebitehnoloogiate abil, kuid Capacitor võimaldab selle viia Androidi ja iOS-i platvormidele. Capacitor annab rakendusele ligipääsu mobiilseadme natiivsetele võimalustele.

Näiteks saab Capacitori pluginatega kasutada seadme kaamerat, asukohta, failisüsteemi, teavitusi ja muid seadme funktsioone. Selline lahendus võimaldab suure osa rakendusest kirjutada platvormist sõltumatult ning vajadusel kasutada natiivseid funktsioone läbi Capacitori.

Seega täidavad tehnoloogiad projektis erinevaid rolle: TypeScript on peamine programmeerimiskeel, Ionic pakub kasutajaliidese komponente ja mobiilirakenduse arendamiseks sobivat raamistikku ning Capacitor ühendab veebipõhise rakenduse mobiilseadme natiivsete võimalustega.

5. Kasutatavad teegid ja raamistikud

Ionic 8.8

Ionic on mobiilirakenduste arendamise raamistik, mis pakub suure hulga valmis kasutajaliidese komponente. Näiteks saab kasutada komponente nagu `ion-button`, `ion-input`, `ion-card` ja `ion-list`.

Ionicu oluline eripära on see, et selle komponendid põhinevad Web Components'i tehnoloogial. See tähendab, et Ionicu komponente saab kasutada ilma Reactita. Käesolevas projektis kasutataksegi Ionicut non-React lahendusena.

Capacitor 8.5

Capacitor on Ionicu rakenduste jaoks mõeldud platvormikiht, mille abil saab veebirakenduse ühendada Androidi ja iOS-i natiivsete funktsioonidega.

Capacitori oluline omadus on pluginapõhine ülesehitus. Vajadusel saab projektile lisada plugina, mis annab rakendusele ligipääsu konkreetsele seadmfunktsioonile. See võimaldab kasutada veebitehnoloogiatel põhinevas rakenduses võimalusi, mida tavalisel veebilehel ei ole.

Web Components

Web Components on veebiplatvormi standarditel põhinev tehnoloogia, mis võimaldab luua korduvkasutatavaid HTML-i komponente. Ionic kasutab oma komponentide loomisel Web Components'i tehnoloogiat.

Selle projekti puhul on oluline, et Ionicu kasutamiseks ei ole vaja Reacti. Komponendid töötavad otse veebitehnoloogiate abil ning seetõttu on võimalik hoida projekti tehniline ülesehitus lihtsam ja kasutada TypeScripti koos Ionicu komponentidega.

6. Kokkuvõte

Projekti tehnoloogiline lahendus on üles ehitatud veebitehnoloogiate ja mobiilirakenduste arendamise ühendamisele. TypeScript annab projektile tugeva tüübisüsteemi ja võimaldab kirjutada struktureeritud JavaScripti põhist koodi. Ionic pakub mobiilirakenduse kasutajaliidese jaoks valmis komponente ning Capacitor võimaldab rakendusel kasutada mobiilseadme natiivseid funktsioone.

Kuna Ionicu komponendid põhinevad Web Components'il, ei ole projekti jaoks vaja kasutada Reacti. Selline lahendus võimaldab kasutada ühte tehnoloogilist baasi nii kasutajaliidese kui ka mobiilirakenduse arendamiseks.

Kasutatud allikad

1. TypeScript Documentation. *TypeScript: JavaScript With Syntax for Types*.
<https://www.typescriptlang.org/docs/>
2. Ionic Documentation. *Ionic Framework Documentation*. <https://ionicframework.com/docs>
3. Capacitor Documentation. *Capacitor Documentation*. <https://capacitorjs.com/docs>
4. MDN Web Docs. *Web Components*.
https://developer.mozilla.org/en-US/docs/Web/API/Web_components
5. MDN Web Docs. *JavaScript Promises*.
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise